

Day 3 - Data Structures: Lists, Tuples, Dicts, Sets

Day 3 — Data Structures: Lists, Tuples, Dicts, Sets

Objectives

- Build real intuition for when to reach for each of the four core containers, in plain terms
- Understand, at a level you could explain to someone else, **why** checking membership in a dict/set is so much faster than in a list
- Get comfortable with slicing — a piece of syntax you'll use constantly from here on
- Learn the list/dict/set methods (a "method" is a function attached to a specific piece of data, called with a dot, like `my_list.append(...)`) you'll reach for daily

Why do we need more than one kind of container?

Yesterday you used lists briefly. Today you'll meet the full family of four built-in "containers" — ways of holding multiple values in one variable — and learn that each one exists because it makes a different trade-off. Choosing the right one isn't just a style preference; it affects how fast your code runs and how clearly it communicates your intent to whoever reads it later (including future-you).

Container	Ordered?	Allows duplicates?	Changeable after creation?	How you look things up	What it's for
<code>list</code>	yes	yes	yes	by position (index)	An ordered sequence of items you expect to add to, remove from, or reorder.
<code>tuple</code>	yes	yes	no	by position (index)	A fixed-size group of values that will never change shape — e.g., "this is always exactly an (x, y) pair."

dict	yes (Python remembers insertion order since version 3.7)	keys: no / values: yes	yes	by key (a name you choose), not position	"Look this up by name," like a real dictionary maps a word to its definition.
set	no	no (automatically removes duplicates)	yes	only "is this in here?" — no positional lookup at all	Fast "have I already seen this?" checks, removing duplicates, and comparing two groups of items.

Don't try to memorize this table right now — the rest of today's lesson explains each row in detail, with examples, and by the end it'll feel natural.

Lists — your everyday, general-purpose container

A list is written with square brackets, values separated by commas:

```
fruits = ["apple", "banana", "cherry"]
```

You already used lists briefly on Day 1 and Day 2. Lists are **ordered** (the items stay in the sequence you put them in) and **mutable** (you can change, add, and remove items after creating the list).

Getting one item out — indexing

Every item in a list has a position number, called an **index**, starting from **0** (not 1 — this trips up nearly every beginner at least once, so say it out loud: the first item is at index 0):

```
fruits = ["apple", "banana", "cherry"]
print(fruits[0])    # "apple" -- the FIRST item
print(fruits[1])    # "banana"
print(fruits[2])    # "cherry"
print(fruits[-1])   # "cherry" -- negative indices count from the END; -1 is the LAST item
print(fruits[-2])   # "banana" -- second-to-last
```

Asking for an index that doesn't exist raises an error:

```
print(fruits[10])   # IndexError: list index out of range
```

This is one of the most common errors you'll see in your first weeks — it means you asked for a position that doesn't exist in that particular list, usually because the list was shorter than you assumed, or because of the classic off-by-one mistake (using **1** when you meant **0**, or going one item too far).

Slicing — getting a *range* of items at once

Slicing lets you pull out a sub-section of a list (or a string!) using the pattern `sequence[start:stop:step]`. All three parts are optional, and there's one rule to memorize: **start is included, stop is NOT included** (this is the exact same "stops just before the number you gave it" behavior you saw with `range()` yesterday — Python is consistent about this on purpose).

```
numbers = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

numbers[2:5]      # [2, 3, 4]          -- start at index 2, stop BEFORE index 5
numbers[:3]       # [0, 1, 2]         -- no start given, so it defaults to the beginning
numbers[7:]       # [7, 8, 9]         -- no stop given, so it defaults to the end
numbers[::2]      # [0, 2, 4, 6, 8]   -- no start/stop, step of 2 means "every other item"
numbers[::-1]     # [9, 8, 7, ..., 0] -- a step of -1 walks backward, reversing the list
numbers[-3:]      # [7, 8, 9]         -- negative indices work in slices too: "the last 3 items"
```

Important beginner gotcha: **slicing always returns a brand new list**, completely separate from the original — unlike plain indexing (`numbers[2]`), which just hands you the single item that was already there. This means `numbers[:]` (an empty slice, meaning "everything") is a common, quick way to make an independent copy of an entire list.

The list methods you'll use constantly

A "method" is just a function that belongs to a particular value, called by writing the value, then a dot, then the method name and parentheses:

```
groceries = ["milk", "eggs"]

groceries.append("bread")    # adds "bread" to the END of the list
print(groceries)             # ["milk", "eggs", "bread"]

groceries.insert(0, "coffee") # inserts "coffee" at index 0, pushing everything else along
print(groceries)             # ["coffee", "milk", "eggs", "bread"]

last_item = groceries.pop()   # removes AND returns the last item
print(last_item)              # "bread"
print(groceries)              # ["coffee", "milk", "eggs"]

groceries.remove("milk")      # removes the first item that EQUALS "milk" (by value, not position)
print(groceries)             # ["coffee", "eggs"]

groceries.sort()              # sorts the list IN PLACE (modifies it directly)
print(groceries)              # ["coffee", "eggs"] (alphabetical here)

numbers = [3, 1, 2]
newly_sorted = sorted(numbers) # returns a NEW sorted list, leaves `numbers` unchanged
print(numbers)                 # [3, 1, 2] -- untouched
print(newly_sorted)            # [1, 2, 3]
```

A very common beginner bug: writing `result = my_list.sort()` and then being confused that `result` is `None`. This happens because `.sort()` is designed to modify the list *in place* and, as a signal of that, deliberately returns nothing (`None`) rather than the list itself — its job is to change `my_list` directly, not to hand you a new value. Whenever you want the sorted result as a usable value without touching the original list, use `sorted(my_list)` instead, which returns a new list and leaves the original alone.

`len(some_list)` (a built-in function, not a method) tells you how many items are in a list — you'll use this constantly:

```
print(len(groceries))    # 2
```

Dictionaries — looking things up by name instead of position

A **dictionary** (or "dict" for short) stores **key-value pairs** — every value is associated with a specific key you choose, and you look values up by that key, not by position. Think of a real paper dictionary: you don't say "give me word #4,281" — you look up a specific *word* and get back its definition. Python dicts work the same way, and the "word" (called the **key**) doesn't have to be text — though it very often is.

```
scores = {"Ana": 90, "Bo": 85, "Cy": 78}

print(scores["Ana"])      # 90 -- look up by KEY, not position
scores["Dee"] = 95        # add a brand new key-value pair
scores["Ana"] = 92        # UPDATE an existing key's value
print(scores)             # {"Ana": 92, "Bo": 85, "Cy": 78, "Dee": 95}
```

Asking for a key that doesn't exist raises an error:

```
print(scores["Zed"])      # KeyError: 'Zed'
```

This is dicts' equivalent of a list's **IndexError** — it means "that key isn't in this dictionary." Since it's extremely common to want "give me the value if this key exists, otherwise give me some default, and don't crash," dicts have a built-in method for exactly that:

```
print(scores.get("Zed"))   # None -- no error, just "nothing found"
print(scores.get("Zed", 0)) # 0 -- your own chosen default instead of None
```

Looping over a dict, as previewed yesterday:

```
for key in scores:          # loops over KEYS by default
    print(key)

for key, value in scores.items(): # .items() gives you BOTH, as pairs
    print(key, value)

for value in scores.values():    # just the values, if you don't need the keys
    print(value)
```

Why is looking something up in a dict so much faster than in a list?

This is a genuinely important idea, and one that interviewers for programming jobs like to ask about, so it's worth understanding, not just memorizing. If you want to check whether "Ana" is a key in a dict with a million entries, Python does **not** have to check all one million keys one by one. Here's why:

A dict is built on something called a **hash table**. When you store a key, Python runs it through a function called **hash()**, which turns the key into a number (imagine it as a fingerprint unique to that value). That number tells Python exactly which "slot" in an internal table to put the entry into. Later, when you look up that same key, Python re-computes its hash and jumps *directly* to that slot — no scanning required, regardless of how many other entries exist. This lookup is, on average, done in "constant time" (programmers write this as **O(1)** — you'll see this notation again) — meaning it takes roughly the same tiny amount of time whether the dict has 10 entries or 10 million.

Contrast this with a list: to check whether a value is *anywhere* in a list, Python has no shortcut — it has to check the first item, then the second, then the third, and so on, until it either finds a match or reaches the end. This is called **O(n)** ("linear time" — the time grows in proportion to how many items there are, *n*). For a small list this difference is invisible; for a large one, it's the difference between instant and noticeably slow.

This is also exactly why dict keys (and set items, below) must be immutable types (numbers, strings, tuples — the immutable types from Day 1), and why you'll get an error if you try to use a list as a key:

```
d = {[1, 2]: "value"}    # TypeError: unhashable type: 'list'
d = {(1, 2): "value"}    # fine -- tuples ARE hashable (since they can't be changed after creation)
```

If Python allowed a mutable list as a key, and you later changed that list, its "fingerprint" (hash) would change too — and the dict would no longer be able to find the slot it originally filed that entry under. Rather than allow a subtle, hard-to-track bug like that, Python simply forbids mutable types as dict keys entirely.

Sets — fast membership checks and "no duplicates allowed"

A **set** is an unordered collection that automatically throws away duplicate values and, like a dict's keys, is backed by that same hash-table trick — so checking "is this item in the set?" is just as fast as a dict lookup, for the same underlying reason.

```
unique_visitors = {"ana", "bo", "ana", "cy"}
print(unique_visitors)    # {"ana", "bo", "cy"} -- the duplicate "ana" was automatically dropped
print("bo" in unique_visitors)    # True -- fast membership check
```

Sets support the same operations you might remember from a math class's "set theory" unit:

```
a = {1, 2, 3}
b = {2, 3, 4}

a | b    # union: everything in EITHER set -> {1, 2, 3, 4}
a & b    # intersection: only what's in BOTH -> {2, 3}
a - b    # difference: in a but NOT in b -> {1}
a ^ b    # symmetric difference: in exactly one of them, not both -> {1, 4}
```

A very practical real-world use: if you have two lists and want to know what they have in common, converting both to sets and using `&` is dramatically faster than writing nested loops to compare every item in one list against every item in the other, especially as the lists grow.

Tuples — a "list" that's locked once created

A tuple looks almost like a list but uses parentheses instead of square brackets, and, crucially, **cannot be changed after it's created** — no appending, no removing, no reassigning an item at a position.

```
point = (3, 4)
point[0] = 99    # TypeError: 'tuple' object does not support item assignment
```

Since a tuple can never change shape, using one is a signal to yourself and anyone reading your code: **"this is a fixed-size record, not a growable collection."** A very common place you'll encounter tuples without necessarily reaching for them on purpose: a function that needs to hand back more than one value actually returns a tuple behind the scenes.

```
def min_and_max(numbers):  
    return min(numbers), max(numbers)  # this is really returning the tuple (min(numbers), max(numbers))  
  
lowest, highest = min_and_max([3, 1, 4, 1, 5])  # "unpacking" the tuple into two separate names  
print(lowest, highest)                        # 1 5
```

That last line — assigning `lowest, highest = ...` — is called **unpacking**: Python takes a tuple (or list) with exactly as many items as you have names on the left, and assigns them in order. You'll use this pattern constantly once you start writing your own functions on Day 4.

Exercises

Open `exercises.py`, fill in each `# TODO`, and run `python exercises.py` to see PASS/FAIL for each one.